

C1.3.2 - Étude comparative des solutions techniques

Projet Spity : comparaison des architectures possibles, sélection de la solution cible et identification des ressources nécessaires.

PROJET

Spity

COMMANDITAIRE

Collectif Altitude Grimpe

CERTIFICATION

Titre RNCP

LIVRABLE

Comparatif technique

Objectif de la compétence

Sélectionner l'architecture technique adaptée en réalisant une étude comparative des solutions techniques, en évaluant les options en termes de sécurité, d'environnements systèmes, de réseaux, d'accessibilité et d'impact environnemental.

Livrable attendu

Une étude comparative des solutions techniques envisagées et une identification des ressources matérielles et techniques nécessaires.

Critères couverts

Avantages, inconvénients, sécurité, systèmes, réseaux, accessibilité, impact environnemental et justification des choix.

Périmètre

Architecture web du MVP : frontend, backend, base de données, hébergement, médias, cartographie, qualité et déploiement.

Principe de décision

Privilégier une solution démontrable, maintenable, sécurisable, compatible RNCP et proportionnée au périmètre MVP.

Critères de comparaison

Sécurité

Protection des comptes, validation des données, secrets, isolation serveur, conformité RGPD et limitation de la surface d'attaque.

Maintenabilité

Typage, structure du code, migrations, documentation, simplicité de débogage et capacité d'évolution après le MVP.

Impact environnemental

Sobriété fonctionnelle, limitation des services, cache, compression, poids des pages et volume de données transférées.

Comparatif des architectures applicatives

Domaine	Solution étudiée	Avantages	Inconvénients / risques	Décision
Architecture générale	Next.js full-stack avec App Router	Frontend et API dans un même projet, démo rapide, typage partagé, cohérent avec Route Handlers.	Monolithe applicatif à surveiller si le projet grossit fortement.	Retenue
Architecture générale	Frontend Next.js + API Express/Nest séparée	Séparation claire des responsabilités, extensible pour grande équipe.	Complexité, double déploiement, plus de configuration réseau et CORS.	Écartée MVP
Frontend	Next.js + React + Tailwind CSS	Stack déjà en place, composants réutilisables, responsive, compatible accessibilité.	Discipline nécessaire sur composants client pour limiter le JavaScript envoyé.	Retenue
Frontend	SPA React + Vite	Très rapide en développement, architecture front simple.	Nécessite une API séparée, SEO et rendu serveur moins intégrés.	Écartée MVP
Backend	Route Handlers Next.js	API colocalisée, faible coût d'infrastructure, adaptée aux formulaires et parcours MVP.	À structurer strictement pour éviter la dispersion de logique métier.	Retenue
Backend	Backend serverless externe	Scalabilité automatique, peu d'administration serveur.	Verrou fournisseur, coûts variables, debugging plus complexe.	À réévaluer V2

Comparatif données et persistance

Domaine	Solution étudiée	Avantages	Inconvénients / risques	Décision
Base relationnelle	MariaDB	Déjà configurée, compatible Docker, adaptée aux relations users, clubs, lieux, posts, events.	Fonctions géographiques moins avancées que PostgreSQL/PostGIS.	Retenue
Base relationnelle	PostgreSQL + PostGIS	Excellent pour géolocalisation avancée et requêtes spatiales.	Migration de stack, coût temps, complexité inutile pour MVP local.	Possible V2
ORM	Drizzle ORM	Typage TypeScript, migrations, contrôle fin SQL, déjà en place.	Demande discipline dans la définition des relations et migrations.	Retenue
ORM	Prisma	Très productif, génération client mature, écosystème large.	Ajout d'un nouvel outil, migration non nécessaire, abstraction plus lourde.	Écartée MVP
Médias	Cloudinary ou S3	Stockage externe, transformations images, allège le serveur applicatif.	Coût variable, conformité, dépendance fournisseur.	Retenue cible
Médias	Stockage local	Simple en démonstration locale, pas de service tiers.	Non adapté production, sauvegardes et volumétrie problématiques.	Démo uniquement

Comparatif services et infrastructure

Domaine	Solution étudiée	Avantages	Inconvénients / risques	Décision
Cartographie	Leaflet + tuiles libres	Open source, coût maîtrisé, adapté à une vue carte MVP.	Qualité et quotas selon fournisseur de tuiles.	Retenue MVP
Cartographie	Mapbox	Rendu riche, API puissantes, géocodage et UX avancés.	Coût, clé API, dépendance fournisseur.	Possible V2
Développement local	Docker Compose	MariaDB/phpMyAdmin reproductibles, proche des contraintes d'exploitation.	Secrets à externaliser, ports à documenter précisément.	Retenue
Déploiement	VPS ou PaaS Node.js + base managée	Déploiement réaliste, contrôle des variables, évolutif après MVP.	Coût mensuel, sauvegardes, supervision à prévoir.	Cible
Déploiement	Hébergement mutualisé classique	Peu coûteux.	Peu adapté à Next.js, Docker, API et tâches serveur modernes.	Écartée

Évaluation transversale

Critère	Architecture retenue	Points forts	Points de vigilance
Sécurité	Next.js API + Drizzle + MariaDB + Zod	Validation serveur, ORM, séparation serveur/client, secrets côté serveur.	Ajouter rate limiting, headers sécurité, contrôle upload et RGPD.
Environnement système	Node.js 20+, Docker Compose, Linux compatible	Reproductible en local, cohérent avec le poste de développement.	Prévoir une cible cloud stable et sauvegardée.
Réseau	Application web + DB + services externes	Architecture simple, peu de flux internes, démo facile.	Sécuriser HTTPS, CORS si API séparée future, quotas cartes/médias.
Accessibilité	React + composants UI contrôlés	Composants réutilisables, labels, focus et contrastes centralisables.	Tester formulaires, carte, médias et composants interactifs.
Impact environnemental	Monolithe web sobre + médias externalisés	Moins de services à maintenir, cache possible, limitation du MVP.	Surveiller images, vidéos, carte, feed infini et requêtes répétées.

Architecture technique retenue

Solution cible MVP

- Next.js App Router pour pages, layout et routes API.
- React et Tailwind CSS pour l'interface responsive.
- Drizzle ORM et MariaDB pour les données relationnelles.
- Zod pour validation des formulaires et entrées API.
- Docker Compose pour MariaDB et phpMyAdmin en local.
- Leaflet pour la cartographie MVP.

Raisons du choix

- Stack déjà en place et maîtrisée.
- Périmètre compatible avec une démonstration RNCP.
- Architecture simple, maintenable et sécurisable.
- Faible coût d'exploitation au démarrage.
- Évolutions possibles vers services dédiés en V2.

Ressources nécessaires

Ressource	Besoin	Configuration minimale MVP	Évolution possible
Poste développeur	Développement, tests, génération migrations et PDF RNCP.	Linux, Node.js 20+, npm, Docker, Chrome headless.	CI GitHub pour automatiser build et checks.
Base de données	Stockage utilisateurs, profils, lieux, posts, events et topos.	MariaDB 11 en Docker avec volume persistant.	Base managée avec sauvegardes automatiques.
Serveur applicatif	Exécution Next.js, API et rendu web.	Local pour développement, PaaS/VPS Node.js pour démo en ligne.	Scalabilité horizontale si usage réel.
Stockage médias	Images de profils, posts et lieux.	Mock ou stockage externe limité en MVP.	Cloudinary/S3 avec transformations et quotas.
Services externes	Cartes, email transactionnel, éventuelles notifications.	Leaflet, fournisseur SMTP simple, notifications hors MVP strict.	Mapbox, push notifications, monitoring applicatif.

Synthèse et décision

L'étude comparative confirme que l'architecture la plus adaptée au MVP Spity est une application full-stack Next.js avec App Router, Route Handlers, MariaDB et Drizzle ORM. Cette solution couvre les besoins du commanditaire tout en limitant la complexité technique et les coûts.

Les solutions plus distribuées ou plus spécialisées, comme un backend séparé, PostgreSQL/PostGIS ou Mapbox, restent pertinentes pour une V2 si la plateforme dépasse le périmètre pilote. Pour la soutenance RNCP, la priorité est de livrer un système démontrable, sécurisé, accessible, maintenable et sobre.

Document produit pour le dossier RNCP - Bloc C1 - Projet Spity.